2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Разрабатываемое программное средство представляет собой мобильное iOS-приложение с серверной частью, функционирующее на основе клиент-серверной архитектуры. Клиентская часть обеспечивает взаимодействие с пользователем, а серверная часть отвечает за обработку запросов, хранение и управление данными.

2.1 Обоснование выбранных технологий для разработки программного средства

2.1.1 Обоснование выбора языка программирования для разработки серверной части

Для реализации серверной части программного средства был выбран язык программирования Python. Данный выбор обусловлен тем, что Python отличается лаконичным и понятным синтаксисом, что упрощает разработку, сопровождение и последующее развитие серверной логики. Важным преимуществом языка в данном случае является наличие развитой экосистемы библиотек и фреймворков, а также модульный подход, позволяющий удобно структурировать серверное приложение и повторно использовать отдельные компоненты. Также хорошо подходит для взаимодействия с внешними компонентами системы, включая базу данных и API. По сравнению с Java использование Python позволяет сократить сложность реализации серверной части и уменьшить объем вспомогательного кода, что важно при разработке относительно компактного клиент-серверного приложения. В сравнении с JavaScript Python обеспечивает более наглядную организацию серверной логики и удобство сопровождения при работе с прикладными сущностями, такими как пользователи, мероприятия и заявки. Это делает его подходящим решением для разработки серверной части мобильного приложения, предназначенного для управления волонтерской деятельностью.

2.1.2 Обоснование выбора фреймворка для реализации API

Для разработки API и организации обработки запросов в серверной части программного средства был выбран фреймворк FastAPI. Его использование является целесообразным, так как данный фреймворк предназначен для создания API на языке Python, поддерживает аннотации типов и включает встроенные средства валидации данных. Это упрощает организацию обмена данными между мобильным приложением и серверной частью, а также снижает вероятность ошибок при обработке запросов.

Также выбор обоснован тем, что преимуществом FastAPI является наличие встроенных средств для описания моделей данных и автоматического формирования документации API. По сравнению с более простыми решениями это делает процесс разработки более удобным и понятным. В то же время FastAPI не требует такого объема настройки, как более сложные фреймворки, что важно при разработке серверной части.

2.1.3 Обоснование выбора реляционной базы данных

Для хранения данных в разрабатываемом приложении была выбрана реляционная база данных PostgreSQL. Выбор данной СУБД обусловлен тем, что структура разрабатываемого приложения включает большое количество взаимосвязанных сущностей, таких как пользователи, мероприятия, заявки, роли и результаты участия волонтеров. Использование PostgreSQL позволяет удобно организовать работу с подобной структурой, обеспечить согласованность данных при одновременном выполнении операций и реализовать хранение информации в форме, удобной для последующей обработки на серверной стороне.

По сравнению с другими реляционными базами данных, PostgreSQL в большей степени подходит для приложений, в которых важны гибкость структуры данных и работа со сложными взаимосвязями между сущностями. MySQL также является реляционной базой данных и поддерживает правила связей между таблицами, однако для рассматриваемого приложения более предпочтительным является решение, ориентированное на работу со сложной моделью данных и дальнейшее расширение структуры системы. SQLite представляет собой встраиваемую базу данных, предназначенную в первую очередь для локального хранения информации.

2.1.4 Обоснование выбора шаблона проектирования для разработки клиентской части

При проектировании клиентской части был выбран архитектурный паттерн MVVM, который обеспечивает разделение пользовательского интерфейса, логики представления и данных. Использование данного паттерна позволяет сделать структуру приложения более понятной и упорядоченной, а также упростить сопровождение и дальнейшее развитие программного средства.

Также использование MVVM сочетается с фреймворком SwiftUI, так как данный подход позволяет связывать представление с состоянием данных и автоматически обновлять интерфейс при изменении модели представления. Это упрощает реализацию клиентской части и делает выбранный паттерн подходящим для разработки мобильного приложения для платформы iOS.

2.1.5 Обоснование выбора языка и фреймворка для разработки клиентской части

Для разработки пользовательского интерфейса были выбраны язык программирования Swift и фреймворк SwiftUI.

Swift является основным современным языком разработки для платформ Apple. Фреймворк SwiftUI использует декларативный подход к построению пользовательского интерфейса, поддерживает автоматическое обновление представлений при изменении данных. Дополнительным преимуществом SwiftUI является наличие встроенных механизмов адаптации интерфейса к различным размерам экранов, а также возможность применения единого подхода при разработке приложений для iOS и iPadOS без необходимости

параллельной реализации представления и логики управления для каждой платформы. Это упрощает сопровождение, внесение изменений и дальнейшее развитие программного средства.

2.2 Структурная организация программного средства

На основании проведённого анализа и сформированных требований была разработана структурная схема ГУИР.400201.311 С1, на которой показана взаимосвязь между основными блоками.

Программное средство можно разделить на следующие функциональные блоки:

– блок аутентификации и авторизации;
– блок управления и хранения данных;
– блок обработки данных;
– блок пользовательского интерфейса;
– блок обработки заявок;
– блок обработки мероприятий;
– блок уведомлений;
– блок координации волонтёров;
– блок формирования рейтинга надежности волонтёров.

2.2.1 Блок аутентификации и авторизации

Блок аутентификации и авторизации отвечает за регистрацию пользователей, их вход в систему, назначение ролей и разграничение прав доступа. В качестве основного метода аутентификации используется JWT, что позволяет безопасно передавать и проверять учетные данные без необходимости хранения сессионных данных на сервере.

Для защиты от несанкционированного доступа в блоке реализована автоматическая проверка токена при каждом обращении к защищённым ресурсам. В случае отсутствия токена или истечения срока его действия пользователь перенаправляется на страницу входа. Это обеспечивает доступ к системе только авторизованным пользователям. После удаления учётной записи возможность повторного входа в систему исключается.

Блок поддерживает верификацию адреса электронной почты с использованием одноразового кода подтверждения, а также восстановление пароля через ссылку, направляемую на зарегистрированный адрес электронной почты пользователя.

Кроме этого, блок поддерживает разграничение прав доступа на основе ролей, что позволяет разделять пользователей по уровням доступа. Это обеспечивает контроль над функциональностью системы, предоставляя разные возможности пользователям разных категорий.

2.2.2 Блок управления и хранения данных

Блок управления и хранения данных предназначен для хранения и структурирования информации, используемой в работе системы. В рамках

данного блока хранятся сведения о пользователях, мероприятиях, заявках, уведомлениях, ролях пользователей, а также данные, связанные с активностью волонтеров и формированием их рейтинга надежности. Данный блок обеспечивает выполнение основных операций над данными, включая их добавление, получение, изменение и удаление. Взаимодействие с ним осуществляется через блок обработки данных, который передает запросы на чтение или обновление информации.

2.2.3 Блок обработки данных

Блок обработки данных представляет собой серверную часть программного средства и обеспечивает прием, обработку и маршрутизацию запросов, поступающих от мобильного приложения. Через данный блок выполняется взаимодействие между клиентской частью, функциональными блоками системы и блоком хранения данных. Результатом его работы является получение, обработка и передача данных, необходимых для выполнения пользовательских и административных операций.

Взаимодействие блока обработки данных с блоком пользовательского интерфейса осуществляется посредством REST API.

2.2.4 Блок пользовательского интерфейса

Блок пользовательского интерфейса обеспечивает взаимодействие пользователя с системой и отвечает за отображение информации. Через данный блок волонтёр, организация или администратор получает доступ к основным функциям программного средства: просмотру мероприятий, подаче заявок, получению уведомлений, работе с личным профилем.

Кроме отображения информации, данный блок обеспечивает ввод и передачу данных, необходимых для выполнения действий пользователя. Взаимодействие блока представления данных с остальными частями системы осуществляется через блок обработки данных, которому передаются запросы пользователя и от которого поступают результаты обработки.

2.2.5 Блок обработки заявок

Для разрабатываемого программного средства важной частью является блок обработки заявок, так как именно он обеспечивает подачу заявок на участие в волонтёрских мероприятиях и их дальнейшее рассмотрение. Через данный блок пользователь может отправить заявку, а организатор – просмотреть, подтвердить или отклонить её.

Блок обработки заявок получает соответствующие команды через блок обработки данных, после чего обращается к блоку управления и хранения данных для сохранения или обновления сведений о заявках. При изменении статуса заявки информация также может передаваться в блок уведомлений, чтобы пользователь своевременно получил сообщение о результате.

2.2.6 Блок обработки мероприятий

Блок обработки мероприятий предназначен для работы с данными о

волонтерских мероприятиях. С его помощью осуществляется создание, редактирование, удаление и просмотр мероприятий, а также управление их основными параметрами, включая название, описание, дату и время проведения, место, категорию, количество участников и другие характеристики.

Взаимодействие данного блока с остальными частями системы осуществляется через блок обработки данных, от которого поступают запросы на выполнение операций с мероприятиями. После обработки соответствующих запросов результаты передаются обратно в блок обработки данных для последующего использования в системе.

2.2.7 Блок уведомлений

Блок уведомлений предназначен для информирования пользователей о событиях, происходящих в системе. Через него отправляются уведомления о регистрации, изменении статуса заявки, создании нового мероприятия, изменении условий участия, отмене мероприятия и других значимых действиях.

Блок уведомлений получает сведения о значимых событиях из блока обработки данных. На основании этих сведений формируются уведомления, передаваемые пользователю через интерфейс приложения.

2.2.8 Блок координации волонтёров

Блок координации волонтеров в ходе проведения мероприятия предназначен для организации и контроля деятельности участников в процессе выполнения волонтерских задач. С помощью данного блока осуществляется распределение волонтеров по группам, а также оперативное обновление информации о ходе мероприятия. Работа данного блока основана на данных, поступающих из блока обработки данных. На их основе выполняются распределение волонтеров, назначение задач и обновление информации об их текущем статусе. Сформированные результаты передаются в систему для дальнейшего использования при организации и контроле хода мероприятия.

2.2.9 Блок формирования рейтинга надежности волонтёров

Блок формирования рейтинга надежности волонтеров предназначен для количественной оценки надежности участников на основе накопленной информации об их активности в системе. В рамках работы данного блока учитываются сведения о поданных заявках, фактах участия в мероприятиях, регулярности активности и других показателях, характеризующих вовлеченность и дисциплинированность волонтера.

На основании поступающих данных для каждого пользователя формируется рейтинг, отражающий степень его надежности как участника волонтерской деятельности. Полученные результаты используются при последующей организации мероприятий, распределении волонтеров и отображении информации в системе.

2.3 Выводы по разделу

В результате системного проектирования была разработана структурная схема, на которой отражено взаимодействие основных блоков системы. Каждый из блоков выполняет строго определённые функции и взаимодействует с остальными через блок обработки данных, что обеспечивает чёткое разграничение ответственности и упрощает сопровождение системы. Также были определены технологии, используемые при разработке программного средства:

– для разработки клиентской части Swift и SwiftUI;

– для разработки серверной части Python и FastAPI;

– база данных PostgreSQL.

Выбранный стек технологий позволяет реализовать клиент-серверное мобильное приложение для организации и управления волонтерской деятельностью, соответствующее требованиям надежности, расширяемости и удобства дальнейшего сопровождения.